

```
# Gra Arkanoid - pokaz gotowego projektu
# grafiki:
# https://drive.google.com/drive/folders/16fuwuD8Fmd3M\_Uq5xATW016AUvNRu\_eM
# - prezentacja krótków działania projektu
```

ARKANOID ETAP I:

```
# - szablon aplikacji z tłem
# - sterowanie platformą
```

```
#-----
```

```
# 1. Podstawowe okno aplikacji
```

```
import pygame
pygame.init()
```

```
SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800
```

```
ekran = pygame.display.set_mode([SZEROKOSC_EKRANU, WYSOKOSC_EKRANU])
zegar = pygame.time.Clock()
obraz_tla = pygame.image.load('images/background.png')
```

```
status_gry = True
while status_gry:
    zdarzenia = pygame.event.get()
    for zdarzenie in zdarzenia:
        if zdarzenie.type == pygame.KEYDOWN:
            if zdarzenie.key == pygame.K_ESCAPE:
                status_gry = False
            # (6 pomoc tłumaczenia) wykrywamy tylko moment wciśnięcia
        elif zdarzenie.type == pygame.QUIT:
            status_gry = False
```

```
    ekran.blit(obraz_tla, (0,0))
```

```
    pygame.display.flip()
    zegar.tick(30)
```

```
pygame.quit()
```

```
# TESTY - powinno wyświetlać się tło
```

```

# 2. Plik Platforma.py =====
# PLATFORMA:
# - sterowana przez gracza,
# - będzie się przesuwać w lewo i prawo
# - nie będzie wyjeżdżać poza ekran
import pygame

SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800

class Platforma(pygame.sprite.Sprite):
    def __init__(self):
        super(Platforma, self).__init__()
        self.obraz = pygame.image.load("images/pad.png")

        self.zresetuj_pozycje() # tworzymy te metode pod spodem

    def zresetuj_pozycje(self):
        self.rect = pygame.Rect(SZEROKOSC_EKRANU/2-70, WYSOKOSC_EKRANU-100, 140, 30)
        # początkowa pozycja platformy

    def ruszaj_platforma(self, wartosc):
        # klawisze A i D
        predkosc = 15
        self.rect.move_ip(wartosc*predkosc, 0)

        if self.rect.left <= 0: # krawędź lewa
            self.rect.x = 0
        if self.rect.right >= SZEROKOSC_EKRANU: # krawędź prawa
            self.rect.x = SZEROKOSC_EKRANU-140 # platforma ma szerokosc 140

# Powrót do głównego pliku
=====
import pygame
# 3. Dodajemy nowy import -----
from Platforma import Platforma
#-----
pygame.init()

SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800

ekran = pygame.display.set_mode([SZEROKOSC_EKRANU, WYSOKOSC_EKRANU])
zegar = pygame.time.Clock()
obraz_tla = pygame.image.load('images/background.png')

# 4. -----
platforma = Platforma()
#-----

```

```

status_gry = True
while status_gry:
    zdarzenia = pygame.event.get()
    for zdarzenie in zdarzenia:
        if zdarzenie.type == pygame.KEYDOWN:
            if zdarzenie.key == pygame.K_ESCAPE:
                status_gry = False
            # (5 pomoc tłumaczenia) wykrywamy tylko moment wciśnięcia
        elif zdarzenie.type == pygame.QUIT:
            status_gry = False

# 5. sterowanie -----
# poza pętlą for żeby można było ruszać płynnie
# wykrywamy czy klawisz jest wciśnięty
keys = pygame.key.get_pressed()
if keys[pygame.K_a]:
    platforma.ruszaj_platforma(-1)
if keys[pygame.K_d]:
    platforma.ruszaj_platforma(1)
#-----

ekran.blit(obraz_tla, (0,0))

# 6. sterowanie -----
ekran.blit(platforma.obraz, platforma.rect)
# TESTY - sterowanie powinno działać
#-----

pygame.display.flip()
zegar.tick(30)

pygame.quit()
#####
#####
ARKANOID ETAP II:
# - dodanie kulki do odbijania
# - mechaniki jej przemieszczania i kolizji z krawędziami i platformą
#-----
# 1. Plik Platforma.py
import pygame
SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800

class Platforma(pygame.sprite.Sprite):
    def __init__(self):
        super(Platforma, self).__init__()
        self.obraz = pygame.image.load("images/pad.png")

        # 1. dodanie flagi (interakcja z kulką)-----
        self.porusza_sie = 0
        #-----
        self.zresetuj_pozycje() # tworzymy te metode pod spodem

```

```

def zresetuj_pozycje(self):
    self.rect = pygame.Rect(SZEROKOSC_EKRANU/2-70, WYSOKOSC_EKRANU-100, 140, 30)
    # początkowa pozycja platformy

def ruszaj_platforma(self, wartosc):
    # klawisze A i D
    predkosc = 15
    self.rect.move_ip(wartosc*predkosc, 0)

    # 2. flaga (interakcja w kulka)-----
    self.porusza_sie = wartosc
    #-----

    if self.rect.left <= 0: # krawędź lewa
        self.rect.x = 0
    if self.rect.right >= SZEROKOSC_EKRANU: # krawędź prawa
        self.rect.x = SZEROKOSC_EKRANU-140 # platforma ma szerokosc 140

    # 3. nowa metoda -----
    def aktualizuj(self):
        self.porusza_sie = 0
        # !! Plik Kulka.py
    # -----

# Nowy plik Kulka.py
=====
# 4. Plik Kulka.py
import pygame
import random
SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800
# obiekt wektor do ruchu kulki
# Czym jest wektor?
# - Ma kierunek (czyli w którą stronę coś się porusza)
# - Ma długość (czyli jak szybko coś się porusza)

# Można o nim myśleć jak o instrukcji dla obiektu:
#     "Idź w tę stronę z taką prędkością."

# skrót do tworzenia wektora
vec = pygame.math.Vector2 # x,y    2D
predkosc = 15

class Kulka(pygame.sprite.Sprite):
    def __init__(self):
        super(Kulka, self).__init__()
        self.obraz = pygame.image.load("images/ball.png")
        # stworzymy te metode pod spodem
        self.zresetuj_pozycje()
        # promień kulki 16
        self.r = 16
        # flaga przegrana - czy kulka spadła na dół ekranu
        self.przegrana = False

```

```

def zresetuj_pozycje(self):
    #środkowe współrzędne ekranu
    self.wspolrzedne = vec(SZEROKOSC_EKRANU/2, WYSOKOSC_EKRANU-140)
    # rect kulki na ekranie
    self.rect = self.obraz.get_rect(center=self.wspolrzedne)
    # wektor przesunięcia
    self.wektor = vec(0, -predkosc)
    # pod jakim kątem poleci kulka
    self.kat_nachylenia = random.randrange(-30, 30)
    self.wektor.rotate_ip(self.kat_nachylenia)
    self.przegrana = False

def aktualizuj(self, platforma):
    # przesuwanie się o 10 pikseli/klatke w wyznaczonym kierunku (wektor)
    self.wspolrzedne += self.wektor
    # synchronizacja pozycji rect z pozycją kulki
    self.rect.center = self.wspolrzedne
# Powrót do plik głównego(teraz będzie 5.)
=====
import pygame
from Platforma import Platforma
# 5. -----
from Kulka import Kulka
# -----

pygame.init()

SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800

ekran = pygame.display.set_mode([SZEROKOSC_EKRANU, WYSOKOSC_EKRANU])
zegar = pygame.time.Clock()
obraz_tla = pygame.image.load('images/background.png')

platforma = Platforma()
# 6. -----
kulka = Kulka()
# -----
status_gry = True
while status_gry:
    zdarzenia = pygame.event.get()
    for zdarzenie in zdarzenia:
        if zdarzenie.type == pygame.KEYDOWN:
            if zdarzenie.key == pygame.K_ESCAPE:
                status_gry = False
        elif zdarzenie.type == pygame.QUIT:
            status_gry = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_a]:
        platforma.ruszaj_platforma(-1)
    if keys[pygame.K_d]:
        platforma.ruszaj_platforma(1)

```

```

# 7. -----
kulka.aktualizuj(platforma)
# -----

ekran.blit(obraz_tla, (0,0))
ekran.blit(platforma.obraz, platforma.rect)

# 7cd. -----
platforma.aktualizuj()
# -----

# 8. -----
ekran.blit(kulka.obraz, kulka.rect)
# TESTY !!!!
# - kulka leci za kazdym razem w innym kierunku
# - ale wylatuje poza krawedzie ekranu i nie widzi platformy
# -----
# 9. Powrót do pliku Kulka.py
pygame.display.flip()
zegar.tick(30)
pygame.quit()
=====
# 9 cd. Plik Kulka.py
import pygame
import random
SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800
vec = pygame.math.Vector2 # x,y    2D
predkosc = 15

class Kulka(pygame.sprite.Sprite):
    def __init__(self):
        super(Kulka, self).__init__()
        self.zresetuj_pozycje()
        self.r = 16
        self.przegrana = False

    def zresetuj_pozycje(self):
        self.wspolrzedne = vec(SZEROKOSC_EKRANU/2, WYSOKOSC_EKRANU-140)
        self.rect = self.obraz.get_rect(center=self.wspolrzedne)
        self.wektor = vec(0, -predkosc)
        self.kat_nachylenia = random.randrange(-30, 30)
        self.wektor.rotate_ip(self.kat_nachylenia)
        self.przegrana = False

    def aktualizuj(self, platforma):
        self.wspolrzedne += self.wektor
        self.rect.center = self.wspolrzedne
        # 10. Powrót i dodanie !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
        self.sprawdz_kolizje(platforma) # tworzymy te metode pod spodem (10 cd)
        # -----

```

```

# 10 cd. -----
def sprawdz_kolizje(self, platforma):
    # krawędzie ekranu
    if self.rect.left <= 0:
        self.wektor.x *= -1 # zmiana zwrotu wektora w osi X
    if self.rect.right >= SZEROKOSC_EKRANU:
        self.wektor.x *= -1 # zmiana zwrotu wektora w osi X
    if self.rect.top <= 0:
        self.wektor.y *= -1 # zmiana zwrotu wektora w osi Y
    if self.rect.bottom >= WYSOKOSC_EKRANU:
        self.przegrana = True # spada poniżej platformy

    # odbicia od platformy
    if self.rect.colliderect(platforma.rect): # jeśli dotknie platformy
        self.wektor.y *= -1 # odwracamy wektor w płaszczyźnie y do góry
        self.wektor.x += platforma.porusza_sie*5 # zmiana kąta i prędkości lekko
zależnie od odbicia

        if self.wektor.x < -predkosc: # ogranicza zwolnienie
            self.wektor.x = -predkosc
        if self.wektor.x > predkosc: # ogranicza przyspieszenie
            self.wektor.x = predkosc

# -----
# 11. POWRÓT DO PLIKU GŁÓWNEGO I TESTY
# - kulka się ładnie odbija od wszystkiego
# - kiedy spada gra się nie kończy, po prostu nie ma kulki

```

```

=====
# 11 cd. Plik główny (dodanie żyć)
import pygame
from Platforma import Platforma
from Kulka import Kulka
pygame.init()

SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800
# 12. życia -----
Zycia = 3
# -----
ekran = pygame.display.set_mode([SZEROKOSC_EKRANU, WYSOKOSC_EKRANU])
zegar = pygame.time.Clock()
obraz_tla = pygame.image.load('images/background.png')

platforma = Platforma()
kulka = Kulka()

status_gry = True
while status_gry:
    zdarzenia = pygame.event.get()
    for zdarzenie in zdarzenia:
        if zdarzenie.type == pygame.KEYDOWN:
            if zdarzenie.key == pygame.K_ESCAPE:
                status_gry = False
        elif zdarzenie.type == pygame.QUIT:
            status_gry = False

```

```

keys = pygame.key.get_pressed()
if keys[pygame.K_a]:
    platforma.ruszaj_platforma(-1)
if keys[pygame.K_d]:
    platforma.ruszaj_platforma(1)

kulka.aktualizuj(platforma)

# 13. Czy kulka spadła-----
if kulka.przebrana:
    Zycia -= 1
    if Zycia <= 0:
        break
    kulka.zresetuj_pozycje()
    platforma.zresetuj_pozycje()
# TESTY!!!!
# - po spadnięciu kulki, rozgrywka zaczyna się od nowa
#-----

platforma.aktualizuj()

ekran.blit(obraz_tla, (0,0))
ekran.blit(platforma.obraz, platforma.rect)
ekran.blit(kulka.obraz, kulka.rect)

pygame.display.flip()
zegar.tick(30)
pygame.quit()
=====
DODANIE NAPISÓW
=====
import pygame
from Platforma import Platforma
from Kulka import Kulka

pygame.init()
# 1. inicjalizacja czcionki-----
pygame.font.init()
# -----
SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800
Zycia = 3
# 2. czcionka -----
czcionka = pygame.font.SysFont('Comic Sans MS', 24)
# -----
ekran = pygame.display.set_mode([SZEROKOSC_EKRANU, WYSOKOSC_EKRANU])
zegar = pygame.time.Clock()
obraz_tla = pygame.image.load('images/background.png')
platforma = Platforma()
kulka = Kulka()

```



```

status_gry = True
while status_gry:
    zdarzenia = pygame.event.get()
    for zdarzenie in zdarzenia:
        if zdarzenie.type == pygame.KEYDOWN:
            if zdarzenie.key == pygame.K_ESCAPE:
                status_gry = False
            elif zdarzenie.type == pygame.QUIT:
                status_gry = False
    keys = pygame.key.get_pressed()
    if keys[pygame.K_a]:
        platforma.ruszaj_platforma(-1)
    if keys[pygame.K_d]:
        platforma.ruszaj_platforma(1)
    kulka.aktualizuj(platforma)
    if kulka.przegrana:
        Zycia -= 1
        if Zycia <= 0:
            break
        kulka.zresetuj_pozycje()
        platforma.zresetuj_pozycje()
    platforma.aktualizuj()
    ekran.blit(obraz_tla, (0,0))
    ekran.blit(platforma.obraz, platforma.rect)
    ekran.blit(kulka.obraz, kulka.rect)
    # 3. wyświetlenie wyniku -----
    tekst = czcionka.render(f"Życia: {Zycia}", False, (255,0,255))
    ekran.blit(tekst, (16, 16))
    # -----
    pygame.display.flip()
    zegar.tick(30)
pygame.quit()

```

```

#####
#####

```

ARKANOID ETAP III:

QUIZ !!!!!!!!!!!!!!!!!!!!!

<https://quiz.giganciprogramowania.edu.pl/wdp-python2-q3>

ETAP III:

- dodanie klocków do zbijania i poziomów

- kolizja kulki z klockami

```
# 1. Plik Klockek.py =====
import pygame
import copy

# bedziemy kopiować klocki
class Klockek(pygame.sprite.Sprite):
    def __init__(self, x, y, hp_klocka):
        super(Klockek, self).__init__()
        self.obraz_oryginalny = pygame.image.load('images/brick.png')
        self.rect = pygame.Rect(x, y, 96, 48)
        self.hp_klocka = hp_klocka # ile razy trzeba trafić klockek żeby go zbić

    def aktualizuj(self):
        maska_z_kolorem = 0
        if self.hp_klocka == 3:
            maska_z_kolorem = (128, 0, 0)
        if self.hp_klocka == 2:
            maska_z_kolorem = (0, 0, 128)
        if self.hp_klocka == 1:
            maska_z_kolorem = (0, 128, 0)
        self.obraz = copy.copy(self.obraz_oryginalny)
        self.obraz.fill(maska_z_kolorem, special_flags=pygame.BLEND_ADD)

#metoda która musi zostać utworzona, żeby zarządzać w grupie
    def update(self):
        self.aktualizuj()

    def uderzenie(self):
        self.hp_klocka -= 1
        if self.hp_klocka <= 0:
            self.kill()

=====
# 2. Przechodzimy do pliku Kulka.py (Kulka_arkanoid3.py)
# - metoda aktualizuj i sprawdz_kolizje
import pygame
import random

SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800

vec = pygame.math.Vector2
predkosc = 15

class Kulka(pygame.sprite.Sprite):
    def __init__(self):
        super(Kulka, self).__init__()
        self.obraz = pygame.image.load("images/ball.png")
        self.zresetuj_pozycje()
        self.r = 16
        self.przegrana = False
```

```

def zresetuj_pozycje(self):
    self.wspolrzedne = vec(SZEROKOSC_EKRANU/2, WYSOKOSC_EKRANU-140)
    self.rect = self.obraz.get_rect(center=self.wspolrzedne)
    self.wektor = vec(0, -predkosc)
    self.kat_nachylenia = random.randrange(-30, 30)
    self.wektor.rotate_ip(self.kat_nachylenia)
    self.przegrana = False

# 2. nowy parametr 'klocki'
def aktualizuj(self, platforma, klocki): #!!!!!!!
    self.wspolrzedne += self.wektor
    self.rect.center = self.wspolrzedne
    self.sprawdz_kolizje(platforma, klocki) #!!!!!!!

# 2. nowy parametr 'klocki' i kolizja z klockami
def sprawdz_kolizje(self, platforma, klocki): #!!!!!!!

    if self.rect.left <= 0:
        self.wektor.x *= -1
    if self.rect.right >= SZEROKOSC_EKRANU:
        self.wektor.x *= -1
    if self.rect.top <= 0:
        self.wektor.y *= -1
    if self.rect.bottom >= WYSOKOSC_EKRANU:
        self.przegrana = True

    if self.rect.colliderect(platforma.rect):
        self.wektor.y *= -1
        self.wektor.x += platforma.porusza_sie*5
        if self.wektor.x < -predkosc:
            self.wektor.x = -predkosc
        if self.wektor.x > predkosc:
            self.wektor.x = predkosc

# kolizja z klockami !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
for klocek in klocki:
    # nastąpiła kolizja
    if self.kolizja_z_klockami(self, klocek): # tworzymy metode pod spodem
        # żeby kod był czytelniejszy,
        # będziemy w niej sprawdzać
        klocek.uderzenie()
        break

```

```

# 3. Kolizja z klockami
def kolizja_z_klockami(self, kulka, klocek):

    # odległość między środkami kulki i klocka w poziomie (x)
    dystans_x = abs(kulka.rect.centerx - klocek.rect.centerx) - klocek.rect.w / 2

    # odległość między środkami kulki i klocka w pozynie (y)
    dystans_y = abs(kulka.rect.centery - klocek.rect.centery) - klocek.rect.h / 2

    if dystans_x < kulka.r and dystans_y < kulka.r:
        if dystans_x < dystans_y:
            self.wektor.y *= -1 # uderzenie z góry/dołu - odwróć y
        else:
            self.wektor.x *= -1 # uderzenie od boków - odwróć x
        return True
    return False

# Powrót do pliku głównego
=====
import pygame
from Platforma import Platforma
from Kulka import Kulka
# 3. -----
from Klocek import Klocek
# -----

pygame.init()
pygame.font.init()

SZEROKOSC_EKRANU = 1024
WYSOKOSC_EKRANU = 800
# 4. aktualny poziom -----
Poziom = 0
# -----
Zycia = 3

czcionka = pygame.font.SysFont('Comic Sans MS', 24)
ekran = pygame.display.set_mode([SZEROKOSC_EKRANU, WYSOKOSC_EKRANU])
zegar = pygame.time.Clock()
obraz_tla = pygame.image.load('images/background.png')

# 5. Poziomy gry -----
# tablica dwuwymiarowa
poziom1 = [
    [0,0,1,1,1,1,1,1,0,0],
    [0,1,1,1,1,1,1,1,1,0],
    [0,1,1,1,1,1,1,1,1,0],
    [0,0,1,1,1,1,1,1,0,0],
    [0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0]
]

```

```

poziom2 = [
    [0,0,1,1,0,0,1,1,0,0],
    [0,1,2,2,1,1,2,2,1,0],
    [0,1,2,3,2,2,3,2,1,0],
    [0,0,1,2,3,3,2,1,0,0],
    [0,0,0,1,2,2,1,0,0,0],
    [0,0,0,0,1,1,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0]
]

poziom3 = [
    [0,0,3,2,1,2,1,1,0,0],
    [0,1,3,2,2,1,2,1,1,0],
    [0,1,3,2,3,2,1,2,1,0],
    [0,1,3,2,3,1,2,1,2,1],
    [0,1,3,2,0,0,0,2,1,0],
    [0,0,2,0,0,0,0,0,0,0],
    [0,0,0,0,0,0,0,0,0,0]
]

klocki = pygame.sprite.Group()
def dodaj_klocki():
    wczytany_poziom = None
    if Poziom == 0:
        wczytany_poziom = poziom1
    if Poziom == 1:
        wczytany_poziom = poziom2
    if Poziom == 2:
        wczytany_poziom = poziom3

    for i in range(10):
        for j in range(7):
            if wczytany_poziom[j][i] != 0:
                klocek = Klocek(32+i*96, 32+j*48, wczytany_poziom[j][i])
                klocki.add(klocek)
dodaj_klocki()
# -----
platforma = Platforma()
kulka = Kulka()

status_gry = True
while status_gry:
    zdarzenia = pygame.event.get()
    for zdarzenie in zdarzenia:
        if zdarzenie.type == pygame.KEYDOWN:
            if zdarzenie.key == pygame.K_ESCAPE:
                status_gry = False
        elif zdarzenie.type == pygame.QUIT:
            status_gry = False
    keys = pygame.key.get_pressed()
    if keys[pygame.K_a]:
        platforma.ruszaj_platforma(-1)
    if keys[pygame.K_d]:
        platforma.ruszaj_platforma(1)

```

```

# 6. Czy poziom ukończony? -----
if len(klocki.sprites()) == 0:
    Poziom += 1
    if Poziom >= 3:
        break
    kulka.zresetuj_pozycje()
    platforma.zresetuj_pozycje()
    dodaj_klocki()
# -----

kulka.aktualizuj(platforma, klocki) # 7!!!!!!

if kulka.przegrana:
    Zycia -= 1
    if Zycia <= 0:
        break
    kulka.zresetuj_pozycje()
    platforma.zresetuj_pozycje()

# 8. Update() dla całej grupy obrazów -----
klocki.update()
# -----

platforma.aktualizuj()
ekran.blit(obraz_tla, (0,0))

# 9. Wyświetlenie poziomu -----
for klocek in klocki:
    ekran.blit(klocek.obraz, klocek.rect)
# -----

ekran.blit(platforma.obraz, platforma.rect)
ekran.blit(kulka.obraz, kulka.rect)

tekst = czcionka.render(f"Życia: {Zycia}", False, (255,0,255))
ekran.blit(tekst, (16, 16))

pygame.display.flip()
zegar.tick(30)

pygame.quit()

```